

Linux Practical Exam — Questions & Model Answers

File Management, Data Retrieval, System Administration, Archiving & Shell Scripting

Instructions:

1. This exam is to be completed entirely in a Linux terminal — do not use graphical tools.
2. Each question is worth 5–10 marks as indicated. Total marks: 141.
3. Model answers (commands / scripts) are provided directly beneath each question.

Part 1 — Initial Setup and File Management

1. In a single command, set up the following nested folder structure inside your home directory: project/data/raw and project/scripts/tools. *[5 marks]*

Answer:

```
mkdir -p ~/project/data/raw ~/project/scripts/tools
```

The -p flag creates all missing parent directories in one go, so both branches of the tree are built with a single command.

2. Inside the project directory, create a new, empty file named readme.txt. *[5 marks]*

Answer:

```
touch ~/project/readme.txt
```

3. Using the vi editor, create a file named log.txt inside project/data/raw and populate it with 20 lines of sample text (line1, line2, line3, ... line20). *[8 marks]*

Answer:

```
vi ~/project/data/raw/log.txt
# Inside vi:
# 1. Press i to enter INSERT mode
# 2. Type the 20 lines, pressing Enter after each:
line1
line2
line3
...
line20
# 3. Press Esc to leave INSERT mode
# 4. Type :wq and press Enter to save and quit
```

Equivalent one-line shortcut (not using vi, for verification): for i in \$(seq 1 20); do echo "line\$i"; done > ~/project/data/raw/log.txt

4. Duplicate the log.txt file, saving the copy as log_backup.txt inside the project/data directory. *[5 marks]*

Answer:

```
cp ~/project/data/raw/log.txt ~/project/data/log_backup.txt
```

5. Relocate the original log.txt file out of the raw folder and into the project/data directory. *[5 marks]*

Answer:

```
mv ~/project/data/raw/log.txt ~/project/data/log.txt
```

6. Using one command, display a detailed listing — including permissions and ownership — of everything inside the project directory and all of its subdirectories. *[7 marks]*

Answer:

```
ls -lR ~/project
```

-l shows the long format (permissions, owner, group, size, date) and -R recurses into every subdirectory.

Part 2 — Data Analysis and Information Retrieval

7. Print only the first ten lines of log.txt to the terminal. *[5 marks]*

Answer:

```
head -n 10 ~/project/data/log.txt
```

8. Print only the final five lines of log.txt to the terminal. *[5 marks]*

Answer:

```
tail -n 5 ~/project/data/log.txt
```

9. Search log.txt for every line that contains the character “1” and display those matching lines. *[6 marks]*

Answer:

```
grep "1" ~/project/data/log.txt
```

10. Display the entire contents of log.txt with the lines printed in reverse order (the last line shown first). *[6 marks]*

Answer:

```
tac ~/project/data/log.txt
```

On systems without tac (e.g. some BSD/macOS shells), the equivalent is: tail -r ~/project/data/log.txt

Part 3 — System and User Administration

11. Identify and display the process ID (PID) of the sshd process. *[6 marks]*

Answer:

```
pgrep sshd  
# Alternative:  
ps -ef | grep sshd
```

12. Display the system's current calendar in the terminal. *[5 marks]*

Answer:

```
cal
```

13. Modify the permissions on project/data/log.txt so that it becomes read-only for every user — owner, group, and others — with no write or execute access for anyone. *[7 marks]*

Answer:

```
chmod 444 ~/project/data/log.txt  
# Alternative (symbolic form):  
chmod a=r ~/project/data/log.txt
```

14. Set permissions on the project/scripts/tools directory so that every user can execute it, but only the owner has read and write access. *[8 marks]*

Answer:

```
chmod 711 ~/project/scripts/tools
```

711 breaks down as: owner = rwx (7), group = x only (1), others = x only (1) — matching "owner: read+write+execute, everyone else: execute only."

Part 4 — Archiving and Compression

15. Package the entire project directory into a compressed tar archive named project_backup.tar.gz, and place it in your home directory. *[8 marks]*

Answer:

```
cd ~  
tar -czvf project_backup.tar.gz project
```

-c creates a new archive, -z compresses it with gzip, -v lists files as they're added, -f names the output file.

16. Without extracting it, view the list of files contained inside project_backup.tar.gz. [6 marks]

Answer:

```
tar -tzvf ~/project_backup.tar.gz
```

-t lists the archive's contents instead of extracting them.

17. Create a new directory named old_data in your home directory, move project_backup.tar.gz into it, and then extract the archive's contents there. [8 marks]

Answer:

```
mkdir ~/old_data
mv ~/project_backup.tar.gz ~/old_data/
cd ~/old_data
tar -xzf project_backup.tar.gz
```

Part 5 — Shell Scripting and Advanced Commands

18. Write a shell script that accepts a directory path as a command-line argument. The script should check whether the directory exists (printing an error and exiting if it doesn't), and otherwise list every file inside it. [10 marks]

Answer:

```
#!/bin/bash
# list_dir.sh

if [ -z "$1" ]; then
    echo "Usage: $0 <directory_path>"
    exit 1
fi

if [ ! -d "$1" ]; then
    echo "Error: Directory \"$1\" does not exist."
    exit 1
fi

echo "Files in $1:"
ls -l "$1"
```

Run it as: sh list_dir.sh ~/project

19. Search the entire filesystem for any file named readme.txt and print out the full paths of every match. [6 marks]

Answer:

```
find / -type f -name "readme.txt" 2>/dev/null
```

2>/dev/null suppresses "Permission denied" errors from directories the current user can't access.

20. Create a shell script that accepts four command-line arguments. It should first print all four arguments on a single line, then print a greeting addressed to the third argument. (Example: running "sh question4.sh Sam John Reeta Seema" should print "Sam John Reeta Seema" followed by "Hello Reeta!".) [10 marks]

Answer:

```
#!/bin/bash
# greet_third.sh

echo "$1 $2 $3 $4"
echo "Hello $3!"
```

Run it as: sh greet_third.sh Sam John Reeta Seema

21. Create a shell script that repeatedly prompts the user to enter a password, using a while loop that keeps asking until the user enters the correct value: "anmke9084". [10 marks]

Answer:

```
#!/bin/bash
# password_check.sh
```

```
correct_password="anmke9084"
entered_password=""

while [ "$entered_password" != "$correct_password" ]; do
    read -sp "Enter password: " entered_password
    echo
done

echo "Access granted."
```

read -sp hides the typed characters and shows the given prompt; the loop exits only once the entered value matches exactly.